Maple Seed Funding Guide

PRODUCT AND SOFTWARE DESIGN DOCUMENT

Maple Seed Funding Guide

From the problem to a first software design: the product for business readers, and its translation into object-oriented software engineering.

Author	Katherine Borges Sato
Version	0.1, working draft
Date	October 30, 2026
Status	Product design complete · software design in progress

An independent, non-commercial learning project. Maple Seed Funding Guide is not a Government of Canada website and is not affiliated with, endorsed by or sponsored by any government, financial institution or educational institution. Nothing in this document is legal, financial or tax advice.

Contents

About this document 2

Part A. The product 3

A.1 Project overview 3
A.2 Context and motivation 3
A.3 Ideation 5
A.4 Product thinking: who it serves and what it must do 6
A.5 Main features and screens 7
A.6 Brand and identity 13
A.7 Risks, open items and roadmap 14
A.8 Evidence, assumptions and validation 15

Part B. Software engineering 17

B.1 The software development lifecycle and where we stand 17
B.2 Requirements elicitation: actors and use cases 18
B.3 Analysis: the object model 24
B.4 Analysis: the dynamic model 28
B.5 Architectural design 30
B.6 From design to a working MVP 33
Appendix: glossary and sources 36

About this document

This document records everything decided so far about Maple Seed Funding Guide, from the original problem to a first software design, in one place.

It has two parts:

- **Part A, Product (for business readers).** What the project is, why it exists, the evidence behind it, the ideas that shaped it, what the product must do, the brand, and the risks and roadmap. No technical background is needed.
- **Part B, Software engineering (technical).** How Part A becomes software, following the object-oriented software engineering method: requirements as use cases, an analysis model in UML, and a proposed architecture. Each technical term is explained in plain words the first time it appears.

Status. The product design and a clickable prototype (English and French) are complete. The software design in Part B is a first draft, to be refined before building the MVP (minimum viable product: the smallest version that is useful). Figures in Part A come from federal sources read in October 2026 and should be rechecked before reuse.

PART A

The product

Why the project exists, the evidence behind it, what the product must do, the brand, and the roadmap. Written for business readers.

A.1 Project overview

Maple Seed Funding Guide is one place where Canadian small and medium-sized businesses (SMEs) can find, understand and prepare for the government and other support programs that fit them.

It is a bilingual website (English for Canada, French for Québec). A business owner can browse a directory of programs, answer five short questions to see which programs match their business and why, and keep a personal list that becomes a PDF report to share with an accountant or advisor. The site never applies for funding on anyone's behalf; every program links to its official website.

Item	Detail
Type	Personal, independent, non-commercial initiative
Purpose	Learning by doing: product design and software engineering, and a portfolio case study
Author	Katherine Borges Sato
Affiliation	None. Not affiliated with, endorsed by or sponsored by any government, company or educational institution
Data source	Business Benefits Finder dataset, Innovation, Science and Economic Development Canada (ISED), published under the Open Government Licence – Canada
Languages	English (Canada) and French (Québec) from the first version
Current stage	Design prototype complete; MVP not yet built

The design work lives in two companion artifacts: a design canvas with every screen in both languages, the style guide, the brand record and eleven rounds of logo exploration, and a 20-slide project presentation with speaker notes. This document turns both into prose and adds the software design.

A.2 Context and motivation

Small businesses carry a large share of Canada's economy, support for them exists at scale, yet very few use government financing and awareness of specific programs is low.

Why small and medium businesses matter

Indicator	Figure	Period
Employer businesses in Canada	1.10 million	December 2024
Share that are small (1 to 99 employees)	98.2%	December 2024
Share of private-sector jobs held by SMEs	63.6% (about 8 million people)	2024
Share of private-sector GDP from SMEs	47.2% (small 33.2% + medium 14.0%)	2022

The 47.2% is our sum of two shares the source reports separately. Anything that helps these businesses find support reaches a large part of the economy.

Where support comes from

Support for businesses comes from five kinds of providers, which the product uses as its first filter:

- **Federal government:** grants, loans, tax credits.
- **Provinces and territories:** vouchers, tax incentives.
- **Municipalities:** local programs.
- **Non-profit organizations:** financing, mentorship.
- **Private sector:** banks, accelerators.

The information is public. The federal Business Benefits Finder is even published as open data, which makes a project like this possible. Our working premise is that the hard part is not that information is missing, but finding the right program and knowing whether you qualify.

The problem, in numbers

Finding	Figure
Small businesses that requested government financing in 2024	1% (against 24% for trade credit and 9% for debt)
Government financing requests that were approved	79%
Non-users of the Canada Small Business Financing Program who knew it existed	16% (34% among its borrowers)
Those same non-users aware of any federal financing support program	34% (43% among borrowers)

Two things stand out: programs do say yes when businesses ask, and awareness of specific programs is low. The 34% and 43% figures describe awareness of federal financing programs among one program's non-users and borrowers, not among all businesses.

Reading the numbers honestly

The data supports part of the story, not all of it:

- **It supports:** programs exist at scale and approve most requests; awareness of specific programs is low (16% to 34% depending on the group); the potential audience is large (1.10 million employer businesses).
- **It does not show yet:** 79% of small businesses said they did not need financing in 2024, and only 1% named being unaware of financing sources as their main reason for not seeking it. No national figure measures how hard programs are to find or understand.

The opportunity is therefore for owners who **do** need support. The project assumes that finding and understanding programs is hard for them; it does not claim to have measured it.

A.3 Ideation

The product answers five assumed pain points with three steps, find, understand and prepare, guided by six principles.

The owner's pain: hypotheses to test

These statements come from the project's design thinking, not from a survey. They guide the design and should be checked with real business owners.

1. "I do not know which programs exist for my business."
2. "I cannot tell whether I qualify."
3. "Every program site uses its own words and steps."
4. "I do not know what to prepare before applying."
5. "A wrong guess costs time I do not have."

The proposal: find, understand, prepare

Step	Feature	What it does	Pain it answers
Find	Search by Filters	A directory of programs narrowed by provider, type of support, goal, sector, province, size and more	1, 3
Understand	Find My Support	Five short questions; programs sorted into match, maybe and not a match, with the reason for each	2, 5
Prepare	My list and PDF report	Save programs, tick off a checklist of what to prepare, and download a report to share	4, 5

Every program links to its official website. The guide never applies on behalf of the business.

Six principles behind the product

Principle	What it means in practice
Rules, not AI	Matching follows each program's published criteria. No black box.
Always explained	Every result says why: what fits, what to confirm, what does not.
Honest	No promises and no advice. Every program links to its official site.
Private by design	Nothing is saved unless the user chooses. Built with PIPEDA (the federal private-sector privacy law) and Québec's Law 25 in mind.
Bilingual	English (Canada) and French (Québec) from the first version.
Independent	A non-commercial learning project, clearly not a government website.

Privacy and legal points are design guidance, not legal advice; they need review before any public launch.

A.4 Product thinking: who it serves and what it must do

Before building software, we write down who will use it, what they are trying to get done, and what the product must and must not do. In software engineering this step is called *requirements elicitation*: drawing out and recording what the users and the client actually need. Part B.2 restates it in the formal notation engineers use.

Who uses it

User	Who they are	What they want
Visitor	An owner or manager of a Canadian SME, possibly a newcomer or early-stage founder, browsing without an account	To see quickly which programs could apply to their business
Registered user	The same person after creating a free account	To keep a list, a business profile and a report they can share
Content curator	Whoever loads program data and keeps eligibility rules current (today, the project author). Signs in on the same Log in page, with an invitation-only curator account and a second verification step	To keep the program information accurate and up to date

The third user was not visible in the screens designed so far. Writing the requirements down made it appear: someone has to maintain the program data, and the product needs tools for that.

Scenarios: the product told as short stories

- **Browsing.** Maria runs a 12-person food processing business in Ontario. She opens the site, filters by "Ontario", "Agriculture and agri-food" and "Buy equipment", reads two program pages and follows the link to an official site.
- **Guided matching.** Ahmed, a newcomer, has a two-person IT company in Québec. He answers the five questions in French, sees three matches and two "maybe" programs, each with the reasons, and downloads the checklist.
- **Preparing a meeting.** Maria creates an account, saves four programs and generates a PDF report to take to her accountant.
- **Leaving.** Ahmed decides to stop using the site and deletes all his data with one button, after a confirmation.
- **Keeping data current.** The curator imports the latest Business Benefits Finder file, reviews what changed and updates the rules of a program whose criteria changed.

These scenarios were written by the project author to illustrate the intended experience. They are visionary scenarios (how it could work with the product), not as-is scenarios observed with real business owners. See A.8.

What the product must do

Must have (MVP):

- Let anyone search and filter programs without an account.
- Show a program page with amount, type of support, provider, eligibility summary and a link to the official site.
- Run the five-question Find My Support flow and group results into match, maybe and not a match, with reasons per criterion.
- Show a general checklist of what to prepare.
- Let a user create an account with explicit consent, log in and log out.
- Let a registered user keep a business profile and a list of saved programs.

- Generate a PDF report of the saved list.
- Let a user download their data and delete all of it.
- Let the curator import program data and maintain each program's eligibility rules in an internal curator console that only curator accounts can open, after two-step verification.

Should have (after the MVP): sign-in with Google, personalised recommendations on the profile page, optional owner-profile questions (for programs aimed at specific groups), email updates with opt-in.

Will not do: apply for programs, give legal, financial or tax advice, guarantee eligibility or funding, sell data or show advertising.

Qualities the product must have

These are expectations about *how well* the product works rather than *what* it does (engineers call them non-functional requirements):

- **Bilingual:** every screen, message and report in English and French.
- **Accessible:** usable with a keyboard and screen reader, aiming at WCAG 2.1 AA (the common web accessibility standard).
- **Private:** collects only what each feature needs, with separate consent for sensitive information. Curator tools are protected by role checks and two-step verification.
- **Trustworthy:** an independence notice on every page; every program shows its source and the date it was last checked.
- **Explainable:** every match result can be traced to the rule that produced it.
- **Fast and simple:** the guided questions take about two minutes, and the site works on a phone.

A.5 Main features and screens

The prototype has nine screens, each in English and French, plus a style guide, a brand record and four internal curator screens (English only).

Screen	What the user sees and does
Home	The promise in one line ("Funding and subsidies for Canadian SMEs, made simple"), two paths (browse or be guided), a sample PDF report and a short "how it works"
Search by Filters	A directory with 11 filter groups: offered by, type of support, main goal, amount, sector, province or territory, business size, annual revenue, business structure, owner profile, availability. Each card shows provider, type, amount and region; one click saves it
Program details	Maximum amount, type of support, provider, size limit, description, who can apply, how to apply, and a button to the official website
Find My Support	Five questions (about 2 minutes, no account), then results in three groups with reasons, a checklist, and options to download a PDF or save
About the project	The problem, the goal, the audience, MVP scope and project facts
Log in	Email and password, or Google
Create account and consent	Four steps: account, business, goals and optional owner profile, consents
Profile, list and privacy	Business profile, saved programs, PDF report, recommendations, download or delete my data
Legal notice and privacy	A plain-language notice: what the project is, what is collected and why, rights, retention, email rules and complaints

Curator console (internal)

The curator works in a separate, internal area with its own navy header. It is reached from the normal Log in page: a curator account must pass a second verification step and then lands on the program catalogue. Business owners never see these screens.

Screen	What the curator sees and does
Two-step verification	Enters a 6-digit code from an authenticator app; only accounts with the Curator role reach this step
Program catalogue	Counts of Active, Needs review and Closed programs, the last import date, and the programs to review first
Review an import	The five import steps, counts of new, changed, removed, unchanged and skipped records, a before-and-after view of each change, and Confirm import
Edit and check eligibility rules	Each criterion with its rule and its explanation in English and French, a panel to test the rules on a sample business, and Mark rules as checked

In the prototype, the Log in page has a "Log in as curator (demo)" shortcut that opens this flow.

Prototype screens

The screens below come from the clickable prototype on the design canvas. They use example programs and placeholder data.

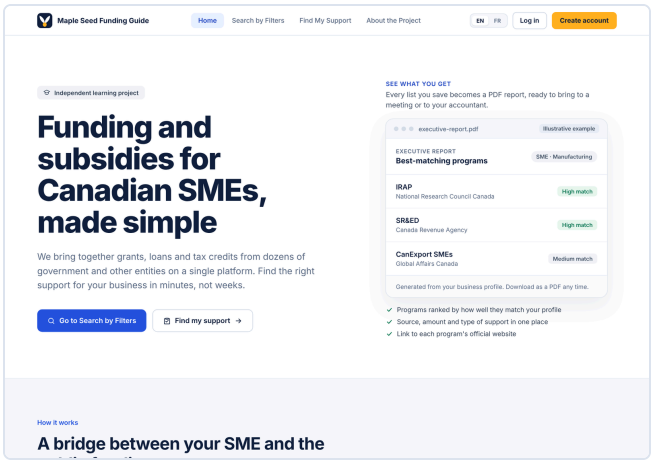


Figure 1. Home (English). The promise in one line, two paths (browse or be guided) and a sample PDF report.

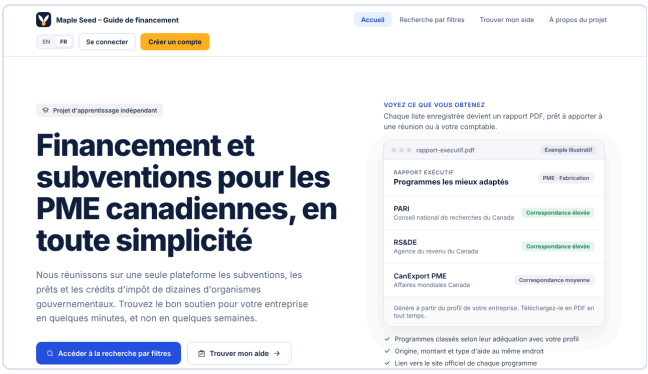


Figure 2. Home (French, Québec). Every screen exists in both languages; the descriptive part of the name is in French.

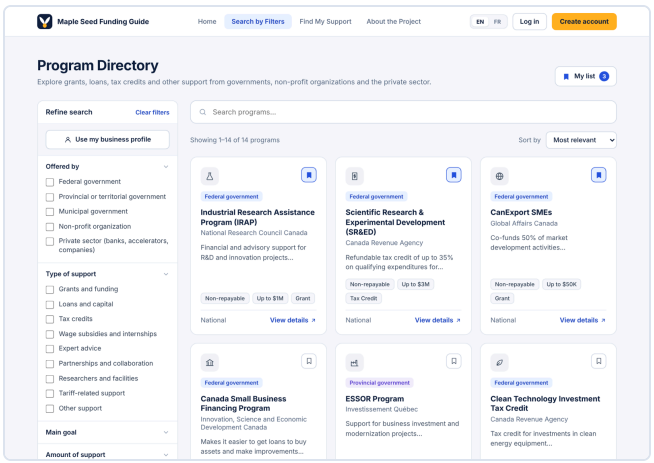


Figure 3. Search by Filters. The filter panel follows the categories of the open government data; one click saves a program.

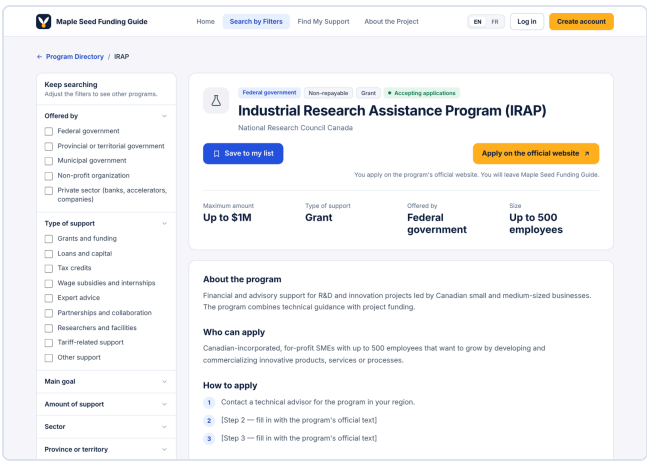


Figure 4. Program details. Amount, type of support, provider and size limit, then who can apply and a button to the official website.

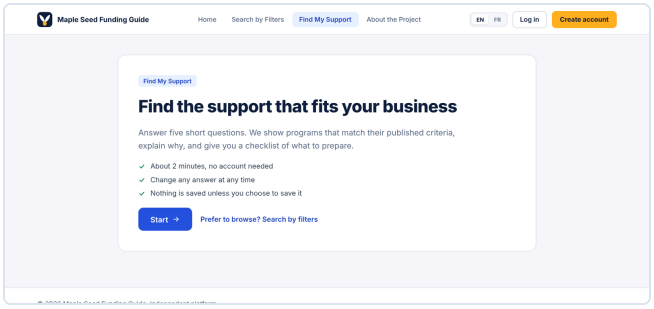


Figure 5. Find My Support, start. Five questions, about two minutes, no account needed.

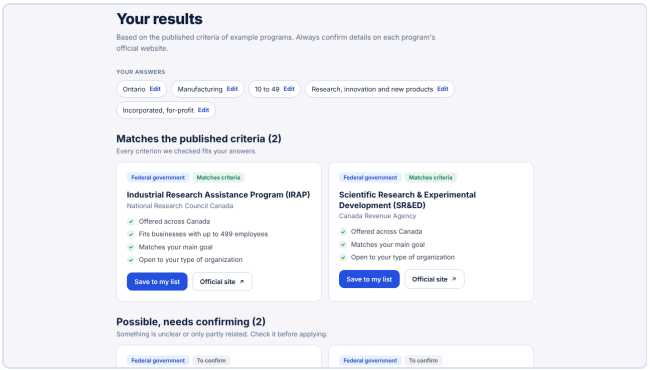


Figure 6. Find My Support, results. The user's answers, then programs grouped by match, each with the reasons per criterion.

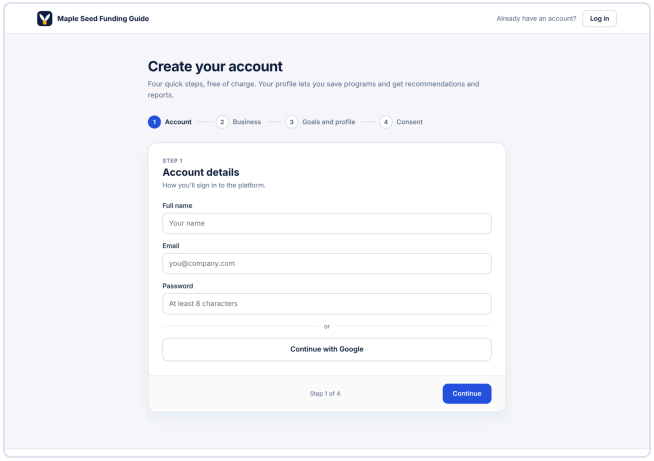


Figure 7. Create account. Four steps (account, business, goals, consent), with sensitive information kept optional.

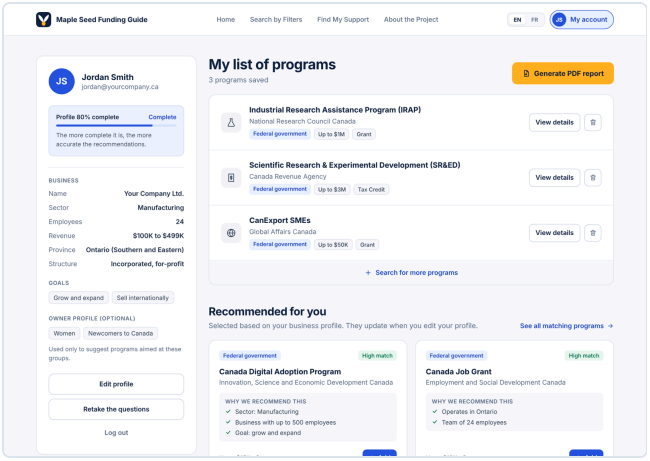


Figure 8. Profile and list. Business profile, saved programs, the PDF report and recommendations.

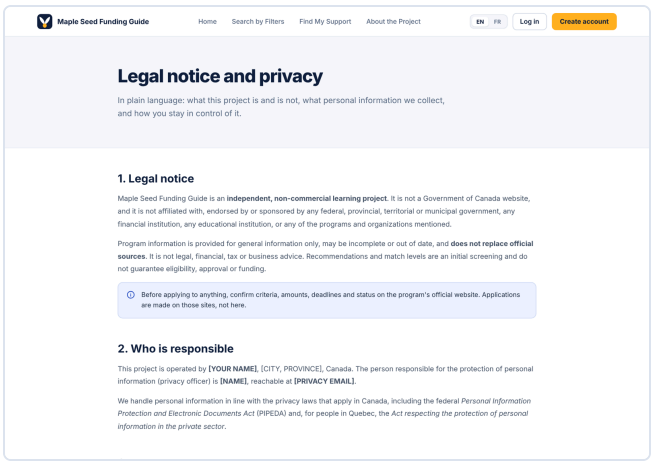


Figure 9. Legal notice and privacy, written in plain language.

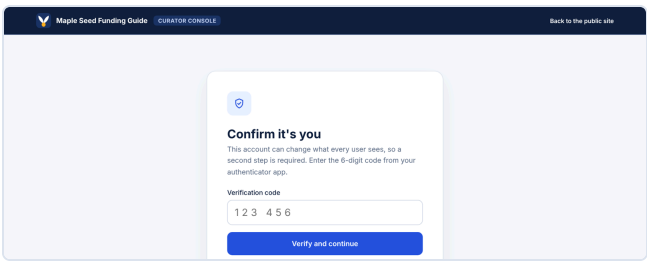


Figure 10. Curator sign-in, second step. Only accounts with the Curator role reach it.

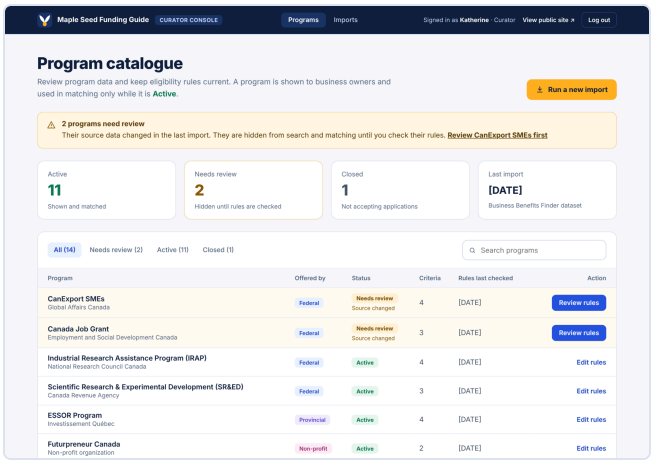


Figure 11. Curator console: the program catalogue, with the programs that need review listed first.

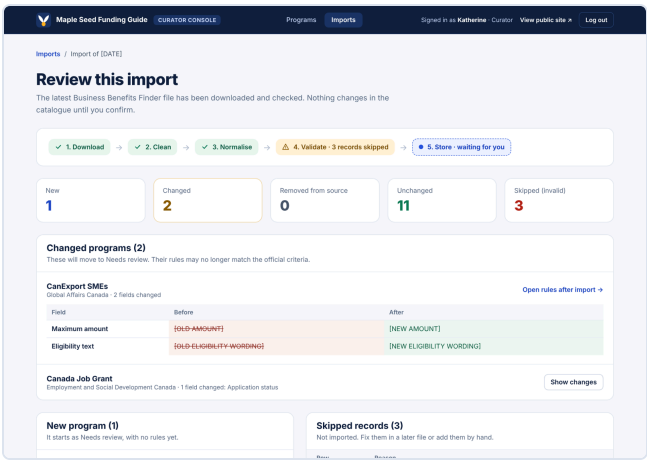


Figure 12. Curator console: reviewing an import before it changes the catalogue.

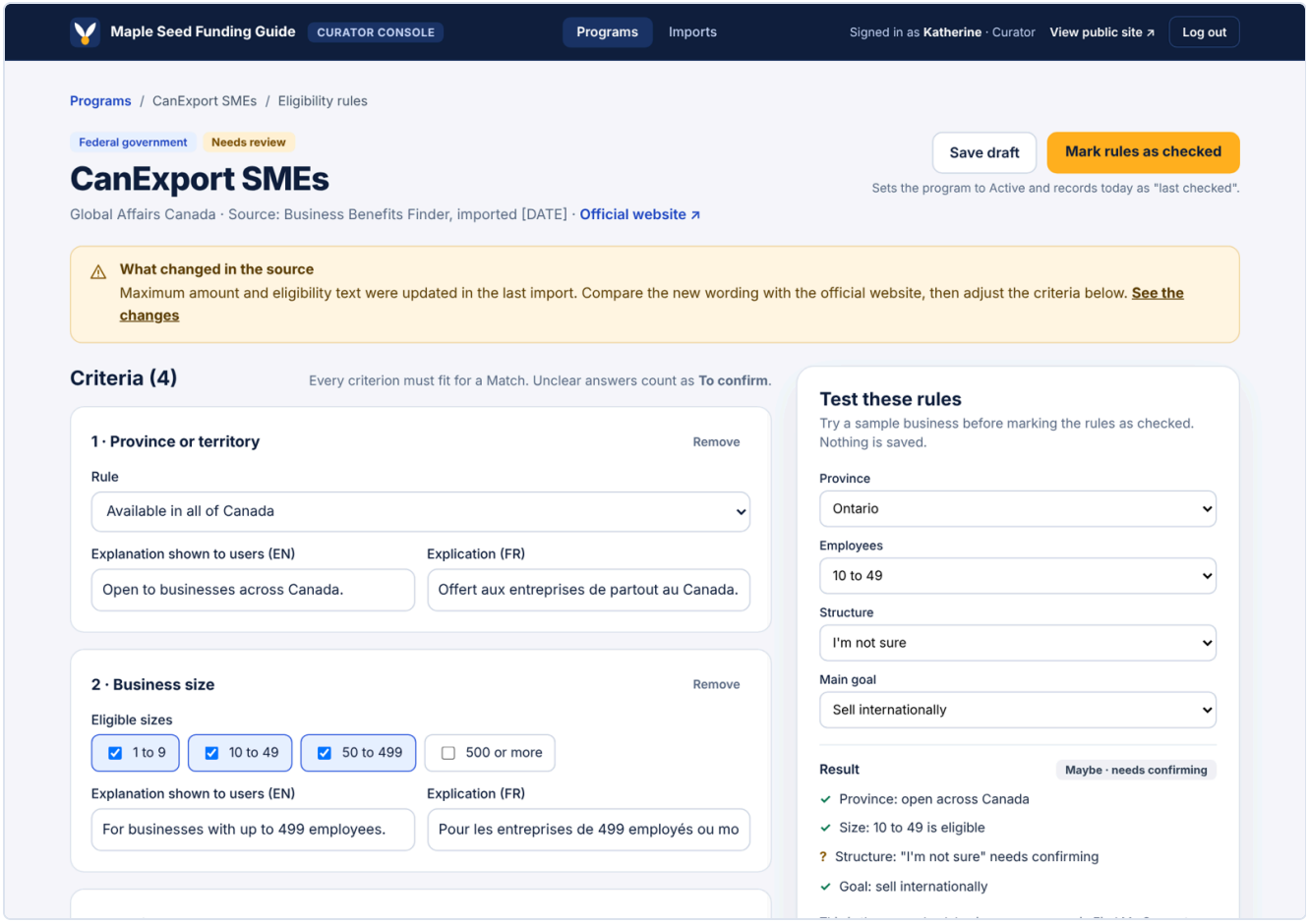


Figure 13. Curator console: editing a program's criteria and testing them on a sample business.

The five guided questions

Question	Options	Why we ask
Where is your business based?	Ontario, Quebec, British Columbia, Alberta, another province or territory	Many programs are limited to one province or territory
What does your business do?	Manufacturing, information technology, agriculture and agri-food, clean energy, services	Some programs target specific sectors
How many employees do you have?	1 to 9, 10 to 49, 50 to 499, 500 or more	Eligibility often depends on size
What is your main goal right now?	Research and new products, sell internationally, hire or train, buy equipment or improve energy efficiency, go digital, increase working capital	Programs fund specific purposes
How is your business set up?	Incorporated for-profit, not incorporated, non-profit, not sure	Some programs are only for incorporated or non-profit organizations

How results are sorted

Each program has a short list of published criteria. For every criterion the system marks the user's answer as **fits**, **to confirm** or **does not fit**. One "does not fit" puts the program under *not a match*; only "to confirm" items put it under *maybe*; otherwise it is a *match*. The quality of the results depends entirely on how accurate and current these rules are.

Example programs in the prototype

The prototype uses 14 example programs to show the design: IRAP, SR&ED, CanExport SMEs, Canada Small Business Financing Program, ESSOR (Investissement Québec), Clean Technology Investment Tax Credit, Canada Digital Adoption Program, Alberta Innovates Micro Voucher, Canada Job Grant, CleanBC Industry Fund, Productivity and Innovation Voucher (Nova Scotia), Manitoba Small Business Venture Capital Tax Credit, Futurpreneur Canada and an example SME line of credit. Their details are simplified and not verified.

Visual style

One typeface (Inter) and colours with fixed roles: midnight blue for headings, cobalt blue for actions and links, amber for the single main call to action per screen, a light blue tint for information that is never clickable, green for confirmation, and red only for destructive actions such as deleting data. Program origin is colour-coded: blue for federal, violet for provincial, cyan for private, pink for non-profit.

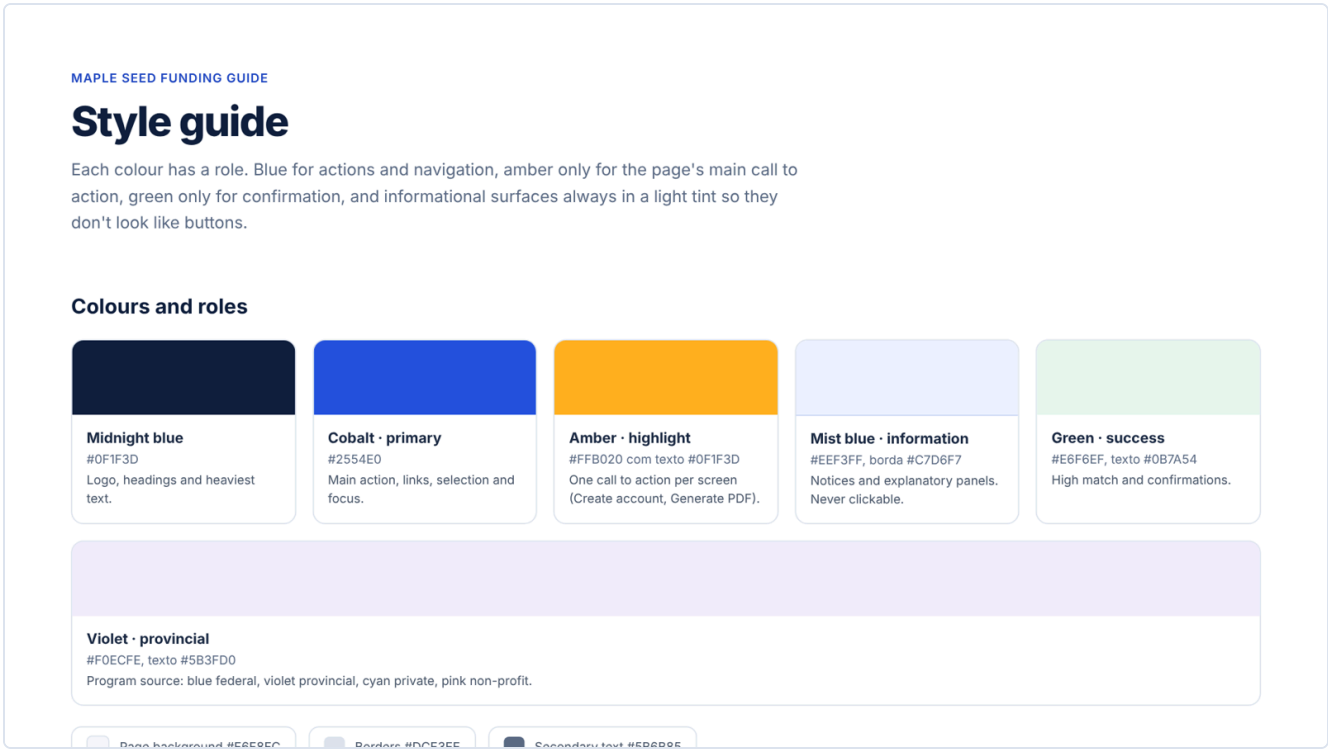


Figure 14. Style guide on the design canvas: colours and their roles, and the button hierarchy.

A.6 Brand and identity

The name is Maple Seed Funding Guide, and the symbol is a pair of maple seed wings joined at one amber seed, chosen after eleven rounds of ideas.

The name

- **Full name:** Maple Seed Funding Guide (short form: Maple Seed). In French: *Maple Seed – Guide de financement*, because Québec's Charter of the French Language asks for the descriptive part in French.
- **Meaning:** "Maple" says Canadian. "Seed" says a small start that can grow, like a young business or a first application. "Funding Guide" says what the site is without promising money.
- **What we avoided:** "Can" or "Canada" in the name, which can make a private site look like a government service, and "Fund", which suggests the site gives or manages money. Names considered along the way include CanBusiness Support, NourishCan, Samara and Maple Seed Project.

The symbol

A maple seed (a *samara*) has a flat wing and often grows in pairs joined at the seed; it spins like a helicopter when it falls. The logo shows two white wings meeting at an amber seed on a navy tile.



Figure 15. The primary logo: two white wings joined at an amber seed, on a navy tile.

Why this one:

- **Two sides, one point:** businesses and programs meeting at a match, without a literal handshake.
- **A start, not a finished thing:** fits small businesses and a learning project.
- **Canadian without the cliché:** avoids the flag's maple leaf, an official national symbol.
- **Clear when small:** one bold shape in two colours, readable at 16 pixels.
- **Sober and trustworthy:** no decoration or gradients; does not look like a bank or a government.
- **Works in one colour:** for print, stamps and PDF reports.

The logo has three versions (primary on navy, on dark backgrounds, one colour). It must not be recoloured, tilted or stretched in the header, placed next to the flag leaf or a crest, or used in a way that hides that the site is independent.

How we got there

Rounds 1 to 11 explored a guide, path and bridge; graphs and threads; handshakes; magnifiers and lanterns; clarity; plants and gardening; Canada without the cliché; NourishCan symbols; five Maple Seed ideas; a single wing; and finally refining the pair. The final mark is Round 11, refinement 1 (it began as Round 9, option 2). The full reasoning, construction grid and usage rules are in the brand and logo record on the design canvas.



Figure 16. A sample of the options explored across the eleven rounds: compass, graph, weave, magnifier, lantern, untangle, trellis, bloom, leaf, watering can, maple seed pair and single wing.

A.7 Risks, open items and roadmap

Design is done; the next phase is a small, well-tested MVP, and several checks remain before anything goes public.

Risks

Risk	How it is handled
Program data is wrong or out of date	The demo uses example data, every program links to its official page, and each program shows when it was last checked
The site looks like a government service	Independence notice on every page, no flag leaf, no "Canada" in the name
The name is already in use	Check domain, trademark and business names before going public
Handling personal data	The demo stores no real personal data; privacy is reviewed before any real use
Scope grows too large	Keep the MVP small: a few programs, done well
"Seed" reads as venture investment (Maple VC is a Canadian seed fund)	"Funding Guide" in the name; monitor reactions
The symbol reads as rabbit ears or a "V" at small sizes	Test it with people who have not seen it
Someone without permission changes program data	Curator accounts by invitation only, a role check on every curator page, and two-step verification

Open items

- [ ] Validate the five pain hypotheses with two or three business owners (informal interviews)
- [ ] Check domains (mapleseed.ca appears taken; try mapleseedguide.ca, mapleseedfunding.ca) with a registrar or CIRA
- [ ] Search "Maple Seed" in the CIPO trademark database and the provincial business registry
- [ ] Have the French wording reviewed by a professional
- [ ] Fill in the placeholders on the About and Privacy pages: privacy officer, contact email, city, hosting location, service providers, retention period, security measures, cookie list, version date
- [ ] Complete the "How to apply" steps on program pages with official text
- [ ] Technology stack chosen on 30 October 2026: Python with FastAPI, React and PostgreSQL (see Part B.5)
- [ ] Add the SavedItem association class for the per-program checklist (object design, B.6)
- [ ] Decide whether a program can have more than one provider (joint programs) during the data validation in B.6
- [ ] Legal review of privacy and legal notices before any public launch

Roadmap

Phase	Content	Status
Design	Screens in English and French, guided matching prototype, brand and logo record, legal and privacy checklist	Done
Software design	Requirements as use cases, analysis model, architecture (Part B of this document)	In progress
MVP	MVP 1 (phases 1 to 7 in B.6): validate the model with 10 to 15 real programs, matching engine with unit tests, database and import, API, and public screens in English and French	Next
After MVP 1	Phases 8 to 10: accounts, saved list and PDF report, curator console, accessibility checks (WCAG), deployed demo, portfolio case study, informal feedback	Then

The scope stays small on purpose: a working, well-documented MVP matters more than covering every program.

A.8 Evidence, assumptions and validation

Everything in this document is grounded in desk research and design reasoning, not in conversations with users: the pain points are hypotheses, and the product is our hypothesis of what could solve them.

What the work is based on

- **Desk research:** public statistics from federal publications (A.2) and reading of online news and articles about Canadian small businesses and funding programs.
- **Design reasoning:** the pain points, the proposal, the prototype and the software models were built from that reading.
- **Not done yet:** no interviews, surveys, observation or usability tests with business owners, accountants or advisors.

How confident we are in each part

Item	Based on	Status
SME statistics (A.2)	Published federal data	Sourced, to be rechecked before reuse
Pain points (A.3)	Desk research and design reasoning	Hypotheses, not validated
Solution: Find, Understand, Prepare	Our proposal to address the pain points	Hypothesis of a solution
Actors (B.2)	Design reasoning and the prototype	Provisional
Scenarios (A.4)	Written by us, visionary only	Provisional, no as-is scenarios yet
Use cases (B.2)	Mainly the prototype screens	Provisional
Analysis model and architecture (B.3 to B.5)	Built on the use cases above	Depends on the use cases being confirmed

How a real project would validate it

In a real project, these steps should be validated before building, so that the actors, scenarios and use cases are confirmed rather than assumed:

1. **Interviews with 3 to 5 SME owners and 1 or 2 accountants or business advisors.** Ask about the past, not about the product: "Tell me about the last time you looked for funding or support for your business. Where did you start? What was hardest? What did you do next?" Avoid "Would you use a site that...?", which people tend to answer yes out of politeness.
2. **Write the stories as as-is scenarios** and compare them with the visionary scenarios in A.4. Keep, change or drop each pain point depending on what people actually describe.
3. **Usability test of the prototype:** give a task ("find a program for your business") and watch where people hesitate, without explaining.
4. **Update the models:** confirm or change the actors, scenarios and use cases, then the class diagram and the architecture. Every use case must keep a traceable reason.

Examples of findings that would change the design: owners who always start with their accountant would suggest a "share my list with my accountant" use case; owners who never want an account would push the saved list into the browser instead of an account.

PART B

Software engineering

How the product becomes software: requirements as use cases, an analysis model in UML, and a proposed architecture.

B.1 The software development lifecycle and where we stand

The project is already in the first stages of the lifecycle: the prototype captured most of the requirements, and this part turns them into an analysis model and an architecture.

Part B follows the method taught in object-oriented software engineering (Bruegge and Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns, and Java*). Software is built through seven activities, each producing a model that feeds the next:

Stage	Question it answers	Main output
Requirements elicitation	What does the client need?	Actors, use cases, non-functional requirements
Analysis	What concepts and behaviour must the system have, seen from the user's side?	Object model (class diagrams, data dictionary) and dynamic model (sequence and state diagrams)
High-level system design	How is the system split into parts, and where do they run?	Subsystems, interfaces, architectural style, deployment
Detailed object design	What exactly does each class look like in code?	Class specifications, design patterns
Implementation	Build it	Source code
Testing	Does it do what the requirements say?	Unit, integration and acceptance tests
Deployment and maintenance	Put it online and keep it working	A running service, updates

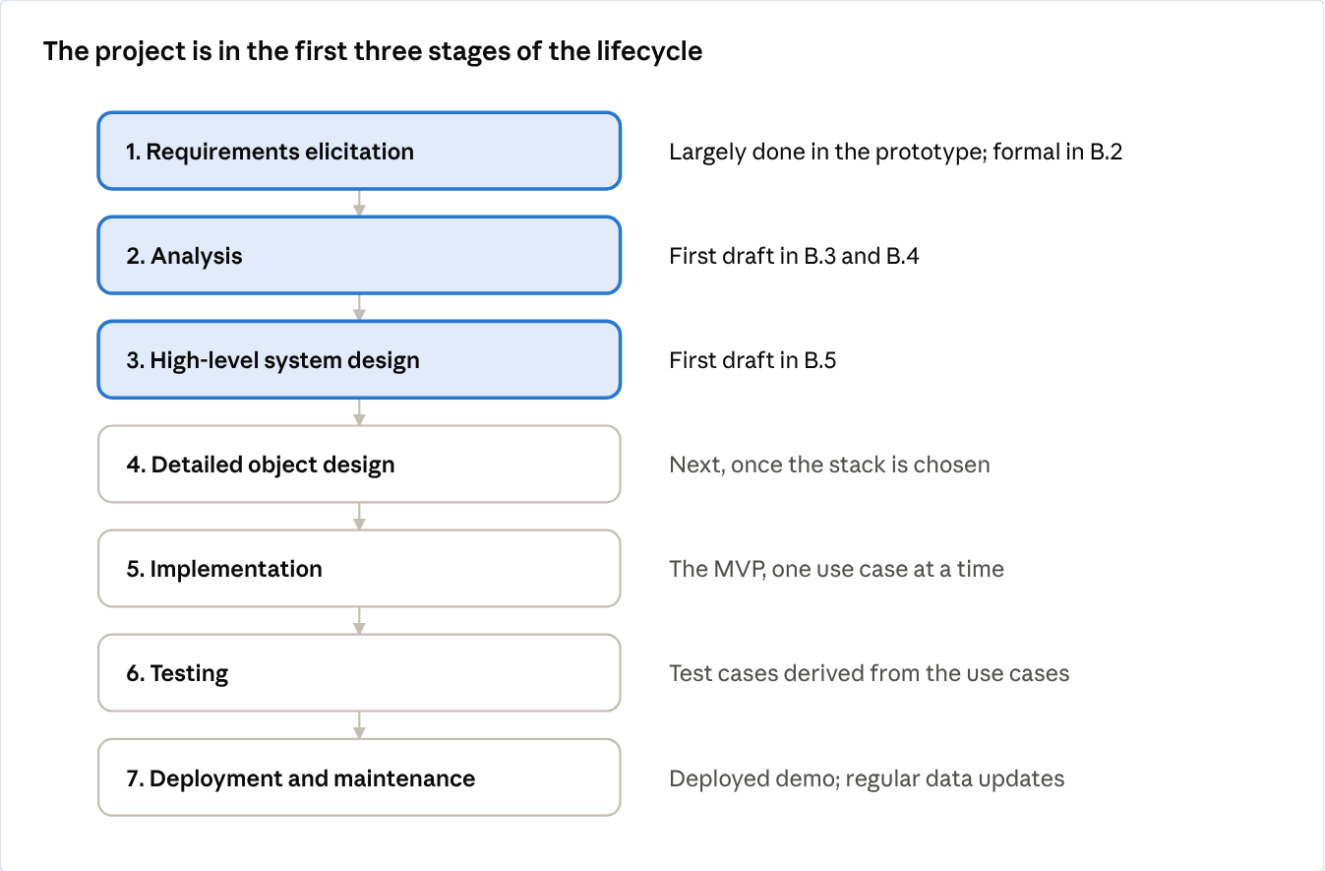


Figure 17. Software development lifecycle · 7 stages, current position highlighted

The stages are not a one-way waterfall. Each pass through them sharpens the earlier models: the clickable prototype was already a way of eliciting requirements, and writing the use cases below revealed a missing user (the content curator).

B.2 Requirements elicitation: actors and use cases

Thirteen core use cases, used by three human actors and supported by external systems, describe everything the MVP must do.

A *use case* is one complete thing a user can achieve with the system, written from their point of view. An *actor* is anyone or anything outside the system that interacts with it: a type of person, or another system. The *system boundary* separates what we build from what we rely on.

Actors

Actor	Kind	Role
Visitor	End user	Anonymous SME owner; searches, reads and runs the guided questions
Registered user	End user	A visitor with an account; inherits everything a visitor can do, plus saving, reports and data controls
Content curator	End user (internal)	Loads program data and maintains eligibility rules; signs in with the Curator role and two-step verification
Business Benefits Finder dataset	External system	Source of program data (open data from ISED)
Google sign-in	External system	Optional identity provider for logging in
Email service	External system	Sends account emails (password reset, confirmations)
Official program website	External system	Where applications actually happen; the guide only links to it

System boundary

Inside the boundary: searching, explaining matches, accounts, saved lists, reports, privacy controls and data maintenance. Outside it: applying for programs, deciding eligibility (only the program can), storing program documents, and anything the official sites do. Keeping applications outside the boundary is a deliberate decision that limits legal risk and scope.

Use case diagram

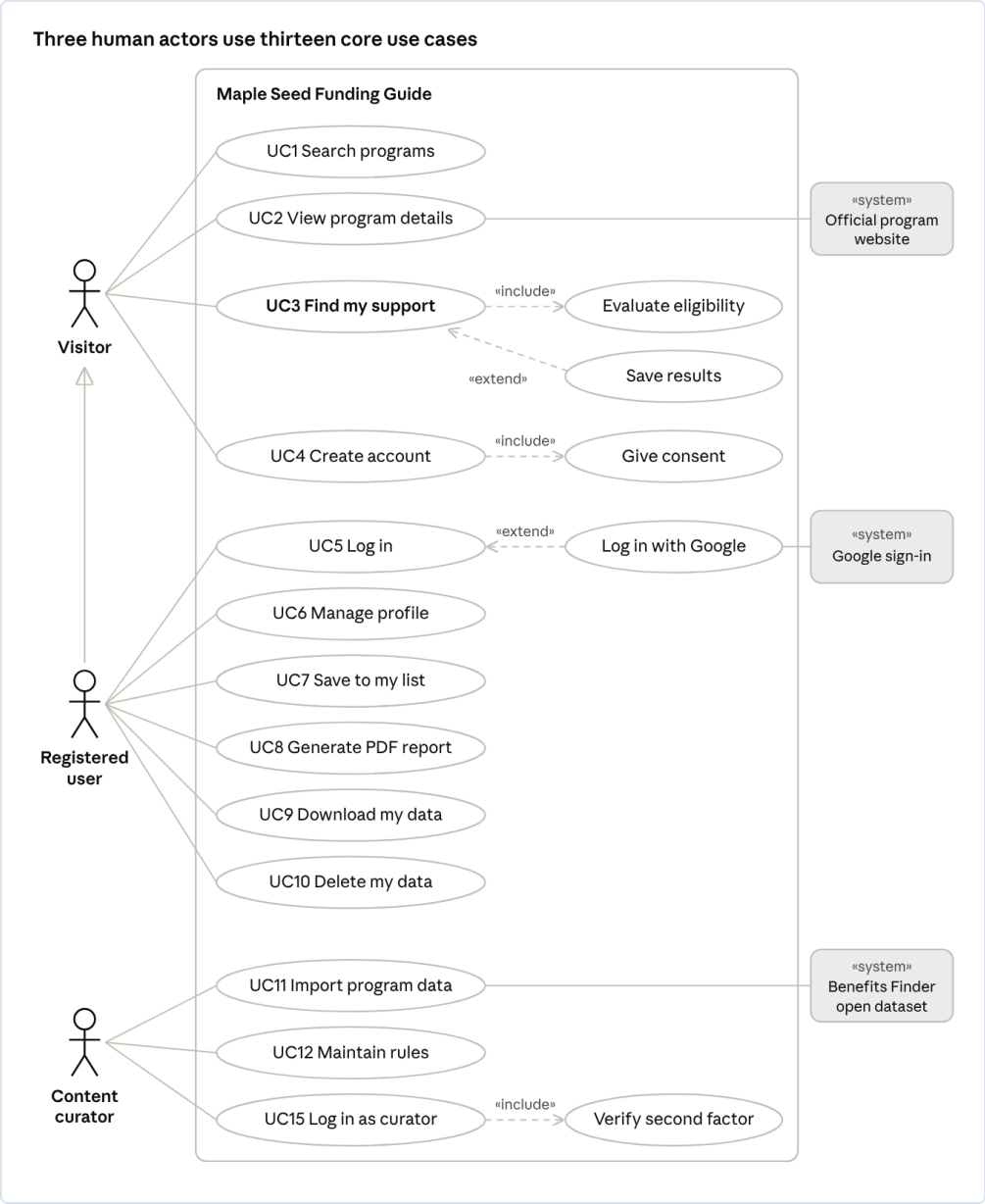


Figure 18. UML use case diagram · MVP scope

How to read it: a solid line means the actor takes part in the use case. The hollow triangle means a registered user is a kind of visitor (inheritance), so every visitor use case is also available to them. A dashed arrow marked «include» means the base use case always uses the other one; «extend» means an optional addition that happens only in some cases.

Relationships between actors and use cases

- **Initiate:** the actor starts the use case (the visitor initiates UC3 Find my support).
- **Participate:** the actor or system takes part without starting it (the Benefits Finder dataset participates in UC11; the official website participates in UC2 when the user follows the link).

Relationships between use cases

Relationship	Meaning	In Maple Seed
Include	Behaviour always reused by the base use case	UC3 Find my support includes Evaluate eligibility; UC4 Create account includes Give consent; UC15 Log in as curator includes Verify second factor. UC1 Search also reuses Evaluate eligibility when "Use my business profile" is on
Extend	Optional behaviour added under a condition	Save results extends UC3 when the user chooses to keep them; Log in with Google extends UC5
Inheritance (generalization)	A specialised actor or use case	Registered user inherits from Visitor. UC9 and UC10 could later share a parent "Exercise a privacy right"

Use case list

ID	Use case	Initiated by	Participating	Why it exists	Release
UC1	Search programs	Visitor		Pain points 1 and 3	MVP 1 (Must)
UC2	View program details	Visitor	Official program website	Pain point 3	MVP 1 (Must)
UC3	Find my support	Visitor		Pain points 2 and 5	MVP 1 (Must)
UC4	Create account	Visitor	Email service	Enabler: lets users keep a profile and a list	Phase 8 (Should)
UC5	Log in	Registered user	Google sign-in (extension)	Enabler	Phase 8 (Should)
UC6	Manage business profile	Registered user		Pain point 2 (recommendations from the profile)	Phase 8 (Should)
UC7	Save program to my list	Registered user		Pain point 4	Phase 8 (Should)
UC8	Generate PDF report	Registered user		Pain point 4	Phase 8 (Should)
UC9	Download my data	Registered user		Legal requirement: privacy by design (PIPEDA, Law 25)	Phase 8 (Should)
UC10	Delete my data	Registered user		Legal requirement: privacy by design (PIPEDA, Law 25)	Phase 8 (Should)
UC11	Import program data	Content curator	Business Benefits Finder dataset	Data quality: the Honest principle	MVP 1 (Must, minimal)
UC12	Maintain eligibility rules	Content curator		Data quality: the Honest principle	Phase 9 (Could)
UC13	Get recommendations	Registered user		Pain points 1 and 2	Later (Could)
UC14	Subscribe to program news	Registered user	Email service	Pain point 1	Not planned (Won't)
UC15	Log in as curator	Content curator		Security requirement	Phase 9 (Could)

Every use case needs a reason that can be traced: a user pain point, a principle, a law or a quality requirement. Not every use case answers a pain point directly; account, privacy and curator use cases exist to enable the others or to meet legal and quality needs.

How these use cases were identified. They were derived mainly from the prototype screens (what goal each screen serves), from the principles and the privacy notice (UC9, UC10), and from questions the screens did not answer (who maintains the data: UC11, UC12; how the curator signs in: UC15). This is the reverse of the textbook path (people, then scenarios, then use cases, then screens), so the list is a well-organised first version that still needs to be confirmed with real users (see A.8).

The authenticator app used for two-step verification is not an actor: it never exchanges information with the system. The curator reads the code on the phone and types it in, so the curator is the only actor in UC15.

Use case descriptions

Each use case gets a table with its actors, the conditions before and after, and the steps. Four key ones follow; the rest use the same template.

UC3 Find my support

Field	Content
Initiating actor	Visitor
Entry condition	The visitor is on any page and selects "Find my support"
Flow of events	1. The system shows question 1 of 5 with a short "why we ask". 2. The visitor answers; the system shows the next question. Repeat until all five are answered. 3. The system runs Evaluate eligibility (include) for every active program. 4. The system shows the answers, then programs grouped into match, maybe and not a match, each with a fits, to confirm or does not fit mark per criterion. 5. The system shows the general preparation checklist
Exit condition	The results and checklist are displayed. Nothing is stored unless Save results (extend) runs
Alternative flows	The visitor goes back to change an answer; the results recalculate. The visitor chooses "Start over"
Quality requirements	Five questions, about two minutes; works without an account; available in English and French

UC8 Generate PDF report

Field	Content
Initiating actor	Registered user
Entry condition	The user is logged in and their list holds at least one program
Flow of events	1. The user selects "Generate PDF report". 2. The system collects the business profile and the saved programs with provider, maximum amount, type of support and official link. 3. The system produces the PDF in the user's language, with the independence notice and the date. 4. The system offers the file for download
Exit condition	The user has the PDF. The list is unchanged
Alternative flow	The list is empty: the system explains how to add programs

UC10 Delete my data

Field	Content
Initiating actor	Registered user
Entry condition	The user is logged in and opens Privacy and data
Flow of events	1. The user selects "Delete my data". 2. The system explains what will be removed and that it is permanent. 3. The user confirms. 4. The system deletes the profile, list and consents, closes the account and logs the user out
Exit condition	No personal data of that user remains; a confirmation page is shown
Alternative flow	The user cancels at step 3: nothing changes

UC11 Import program data

Field	Content
Initiating actor	Content curator
Participating actor	Business Benefits Finder dataset
Entry condition	A new version of the dataset is available
Flow of events	1. The curator starts an import. 2. The system downloads the file. 3. The system cleans, normalises and validates each record. 4. The system compares it with the stored programs and lists new, changed and removed programs. 5. The curator reviews and confirms. 6. The system stores the programs and their "last checked" date; changed programs are flagged for rule review (UC12)
Exit condition	The catalogue reflects the dataset; changed programs are marked "needs review"
Alternative flow	Invalid records are reported and skipped; the import can be cancelled before step 6

UC15 Log in as curator

Field	Content
Initiating actor	Content curator
Entry condition	The curator has a curator account, created by invitation, and is on the Log in page
Flow of events	1. The curator enters email and password. 2. The system sees the Curator role and asks for a second factor (include: Verify second factor). 3. The curator enters the 6-digit code from an authenticator app. 4. The system opens the program catalogue in the curator console
Exit condition	The curator is signed in and can use UC11 and UC12
Alternative flows	Wrong code: the system asks again and locks the account after several failures. The curator uses a backup code. An owner account that opens a curator page is refused
Quality requirements	Role check on every curator page; curator accounts are never created through public sign-up

Non-functional requirements

Category	Requirement
Usability	Find My Support completes in about two minutes; one main call to action per screen
Accessibility	Target WCAG 2.1 AA: keyboard use, screen-reader labels, 4.5:1 text contrast
Localisation	Every text, email and PDF in English (Canada) and French (Québec); the language can be switched on any page
Privacy and security	Minimum data per purpose; separate express consent for sensitive owner-profile data; encryption in transit; password hashing; delete and export on request (PIPEDA, Québec Law 25); role-based access to the curator console, invitation-only curator accounts and two-step verification
Reliability of content	Every program shows its source and last-checked date; every match result is traceable to the rule that produced it
Performance	Search and match results return in under one second for the MVP catalogue
Supportability	Rules are data, editable without changing code; core logic covered by unit tests
Legal and licensing	Attribution under the Open Government Licence – Canada; independence notice on every page; opt-in email per Canada's Anti-Spam Legislation (CASL)

B.3 Analysis: the object model

Nine core classes describe the things the system knows about; the heart of the design is a Program that owns a set of EligibilityCriteria, each able to evaluate a BusinessProfile.

The *object model* lists the concepts of the problem (classes), what each one knows (attributes), what it can do (operations), and how they relate. It is drawn as a UML class diagram. At this stage it describes the problem, not the code: names are business words, not database tables.

Three kinds of analysis objects

Kind	Purpose	Maple Seed examples
Entity objects	Long-lived information the system keeps	Program, Provider, EligibilityCriterion, User, BusinessProfile, Consent, SavedList
Boundary objects	Where actors interact with the system	SearchPage, ProgramPage, QuestionnaireForm, ResultsPage, PdfReport, ImportScreen, CuratorConsole, VerificationForm, BenefitsFinderGateway
Control objects	Coordinate one use case	MatchingController (UC3), ReportController (UC8), ImportController (UC11), AccountController (UC4, UC5, UC10, UC15)

The diagram below shows the entity objects, which carry the domain knowledge.

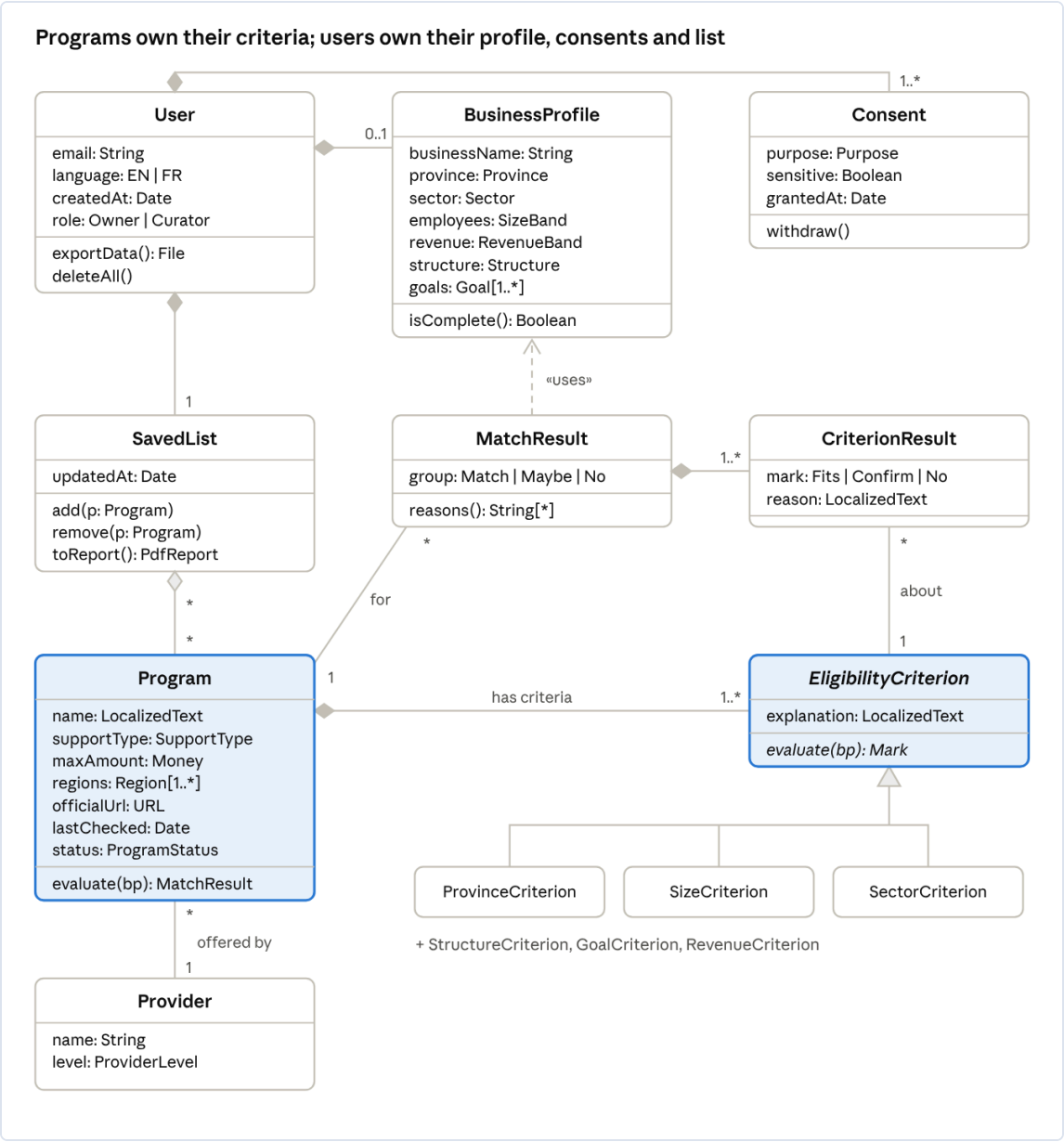


Figure 19. UML class diagram · analysis object model, entity classes

How to read the relationships

Notation	Meaning	Where it appears
Filled diamond (composition)	The part cannot exist without its whole and is deleted with it	A Program's criteria; a User's profile, consents and list; a MatchResult's criterion results
Hollow diamond (shared aggregation)	The whole groups parts that also live on their own	A SavedList holds Programs, but a Program exists regardless of any list, and many lists can hold the same Program
Hollow triangle (inheritance)	A specialised class is a kind of the general one	ProvinceCriterion, SizeCriterion, SectorCriterion and others are kinds of EligibilityCriterion
Numbers at line ends (multiplicity)	How many objects take part	A User has 0..1 BusinessProfile; a Program has 1..* criteria; a Provider offers * (many) programs
Dashed arrow	One class depends on another without owning it	A MatchResult is computed using a BusinessProfile (or the anonymous answers, which have the same shape)

Directionality. Associations are navigable in the direction the use cases need: a SavedList knows its Programs, but a Program does not need to know which lists contain it. A CriterionResult points to the criterion it explains. Keeping links one-way lowers coupling.

Why inheritance for criteria

Each type of criterion checks a profile differently: a province criterion compares regions, a size criterion compares employee bands, a goal criterion looks for overlap between goals. Making them subclasses of an abstract EligibilityCriterion, each with its own *evaluate(profile)*, lets the matching engine treat them all the same way (polymorphism) and lets a new type be added without changing the engine. Rules stay data: the curator sets the values ("Ontario only", "up to 500 employees"), and the subclass supplies the logic.

Why each concept became a class

A noun from the use cases is only a candidate. It became a class when it showed several of these signals:

Signal	Question	Example
Own data	Do several attributes travel together?	BusinessProfile: province, sector, size, structure, goals
Own responsibility	Is there behaviour that does not fit an existing class?	SavedList.toReport()
Identity or sharing	Is it "that one", referred to from many places?	Provider, shared by many programs
Own lifecycle	Is it created, changed or deleted at different times from the others?	Consent is granted and withdrawn on its own
Rules to protect	Is there a rule that must be enforced in one place (encapsulation)?	SavedList.add() refuses duplicates and updates updatedAt

Behaviour goes to the class that holds the data it needs: isComplete() belongs to BusinessProfile, not to a separate checker class. The same encapsulation reasoning explains the small value types used for attributes, such as Money (always carries its currency) and LocalizedText (always holds the English and French versions).

Decisions on specific classes

- **Provider.** A class rather than a text attribute of Program, because it has its own data (name, level), is shared by many programs and is used on its own by the "Offered by" filter. The name is kept and is defined in the data dictionary as the entity that offers a program. In code it may become ProgramProvider, to avoid confusion with the identity provider used for Google sign-in. The link to Program is a plain association, not aggregation or composition: a program is not part of its provider, and neither is deleted with the other. Each program has exactly one provider for now. Joint programs (for example, funded by one entity and delivered by another) would need 1..* or an intermediate class; this is an open question for the data validation in B.6.
- **SavedList.** A class rather than a list attribute of User, because it has its own state (updatedAt), its own operations (add, remove, toReport) and is a concept users see ("My list"). Keeping it out of User keeps User cohesive. Known gap: the checklist of what to prepare belongs to each pair of list and program, so object design should add a **SavedItem** association class (savedAt, checklist) on the SavedList–Program link.
- **CriterionResult.** Separate from MatchResult because a MatchResult holds one verdict per criterion, and each verdict has its own mark, reason and link to its criterion. Folding them into MatchResult would need parallel lists kept in the same order, a sign of a missing class. Separate from EligibilityCriterion because the criterion is the fixed rule, and the result is that rule's verdict for one profile: one rule produces many verdicts.
- **MatchResult.** Answers "does this program fit this business?" for one program. It is created by Program.evaluate(profile), holds the criterion results and derives its group from them. The grouping rule is a logical AND, not a vote: one Does not fit gives Not a match however many criteria fit; no Does not fit but at least one To confirm gives Maybe; all Fits gives Match. Ranking matches against each other is a separate question, outside the MVP. MatchResult is transient: it is recomputed from the profile and the current rules and never stored, so a rule corrected by the curator applies at the next evaluation.

From relationships to code and database

Relationship	In code	In the database	Example
Association	A field holding a reference to an object created elsewhere	Foreign key, no cascade	Program.provider → program.provider_id NOT NULL
Shared aggregation	Same as association; the difference is in meaning only	Link table, no cascade	SavedList and Program → saved_item
Composition	The whole creates its parts, which are deleted with it	Foreign key with ON DELETE CASCADE	A Program's criteria
Dependency	Used as a method parameter or local variable, not stored	Nothing	A MatchResult computed from a BusinessProfile
Inheritance	Subclasses implement an abstract method	One table with a type column, or a JSON column (decided in object design)	EligibilityCriterion subclasses

Navigability decides which side holds the field. A Program knows its Provider; a Provider does not keep a list of programs, which are found through the repository instead (for example, findByProvider).

Data dictionary

Term	Definition
Program	A source of support for businesses (grant, loan, tax credit, voucher, advice) offered by one Provider, with an official website where applications are made
Provider	The entity that offers a program, such as a government department, a Crown corporation, a non-profit organization or a financial institution. The term "entity that offers" is used instead of "agency", which has a specific legal meaning. ProviderLevel is one of federal, provincial or territorial, municipal, non-profit, private
SupportType	Grant, loan, tax credit, voucher, wage subsidy, advisory service, equity or other

Term	Definition
Region	A province or territory, or "all of Canada"
ProgramStatus	Active (accepting applications), Closed, or Needs review (data changed since the rules were last checked)
EligibilityCriterion	One published condition of a program, such as "based in Ontario" or "up to 500 employees", with a plain-language explanation in both languages
Mark	The outcome of checking one criterion: Fits, To confirm (unclear or only partly related), or Does not fit
CriterionResult	One Mark plus the reason shown to the user
MatchResult	The outcome for one program and one profile: Match (all criteria fit), Maybe (no criterion fails, at least one to confirm) or No (at least one does not fit)
BusinessProfile	The business facts used for matching: province, sector, size band, revenue band, structure and goals
SizeBand	1 to 9, 10 to 49, 50 to 499, 500 or more employees
Consent	A recorded permission for one purpose (account, recommendations, sensitive owner profile, email), with its date; it can be withdrawn
SavedList	The programs a registered user has kept; the source of the PDF report
LocalizedText	A text held in both English and French
PdfReport	A generated document with the profile, saved programs, sources, the date and the independence notice
Role	What an account is allowed to do: Owner (a business owner's account) or Curator (can open the curator console). Curator accounts are created by invitation only

B.4 Analysis: the dynamic model

The dynamic model shows behaviour over time: who sends which message to whom during a use case, and which states an object passes through.

The object model says what exists; the dynamic model says what happens. It is drawn from the user's perspective, one use case at a time, and it often reveals operations missing from the class diagram.

Sequence diagram: UC3 Find my support

A sequence diagram reads top to bottom as time passes. Each column is an object (its *lifeline*); each horizontal arrow is a message. Solid arrows are requests, dashed arrows are replies, and boxes marked *loop* repeat.

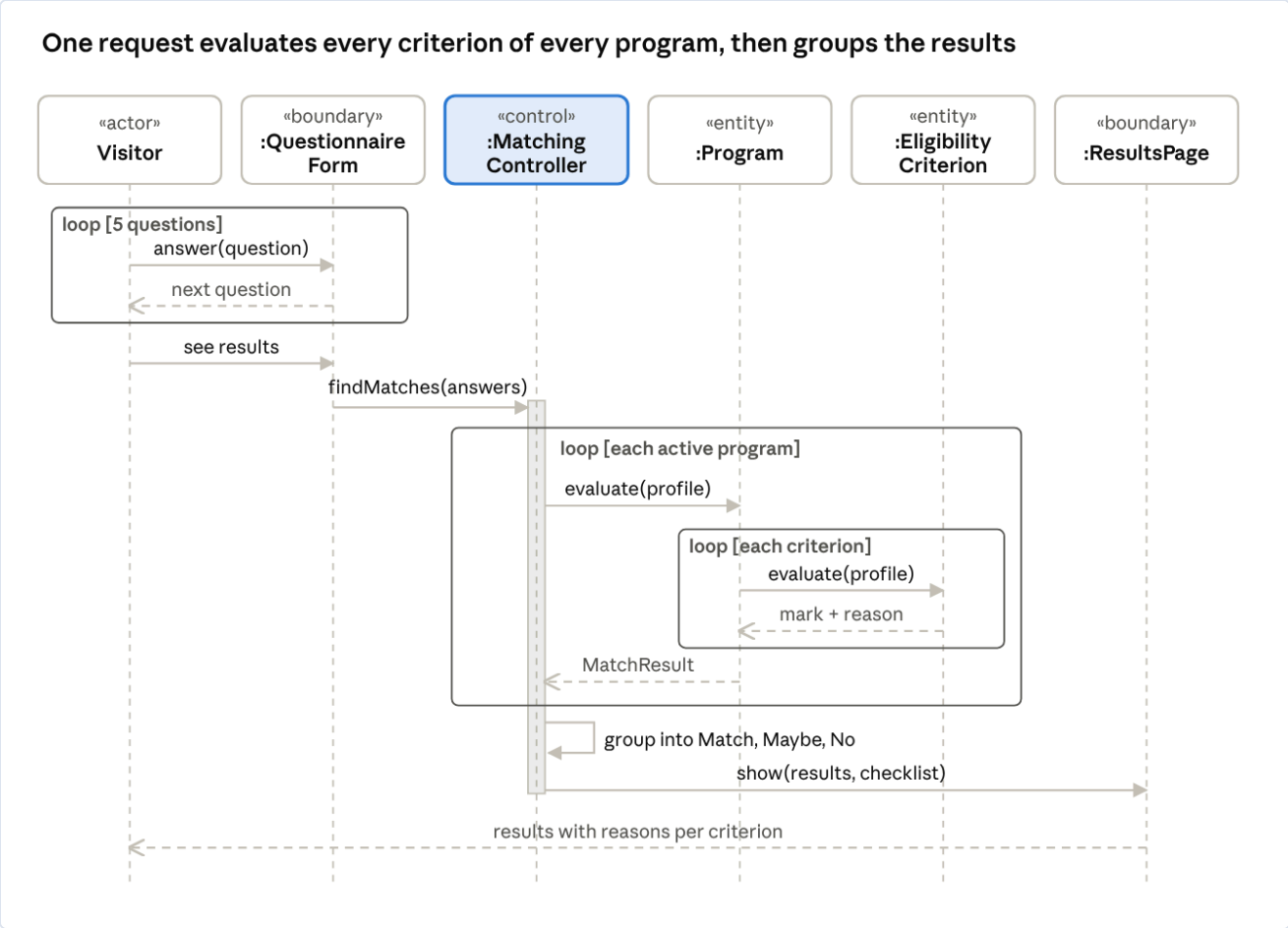


Figure 20. UML sequence diagram · UC3 Find my support

What it shows in plain words: the visitor answers five questions through the form. When they ask for results, the form hands the answers to the matching controller, which asks each active program to evaluate the profile. Each program asks each of its criteria for a mark and a reason, and returns a MatchResult. The controller groups the results into Match, Maybe and No and hands them, with the checklist, to the results page.

What it teaches the design: the controller never checks a rule itself. Programs and criteria do, which keeps the logic in one place and easy to test. The same evaluate operation serves UC1 ("Use my business profile" in search) and UC13 (recommendations).

State machine diagrams

A state machine shows the states one object can be in and the events that move it between them. Labels read *event [condition] / action*.

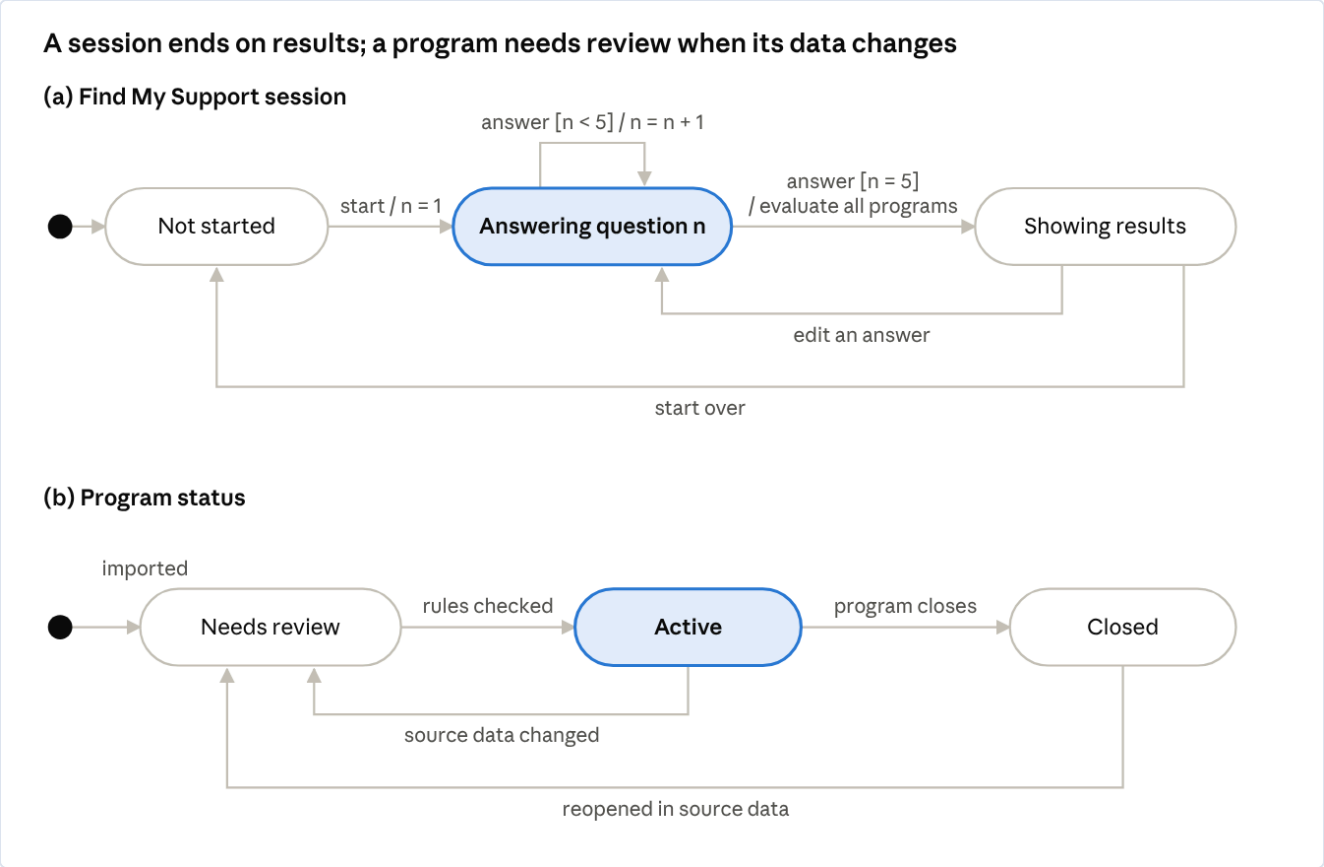


Figure 21. UML state machine diagrams · questionnaire session and program status

- **(a) Find My Support session.** The visitor moves from question to question until the fifth answer triggers the evaluation. From the results they can edit any answer (which returns to that question and recalculates) or start over. Nothing is saved unless they choose Save results.
- **(b) Program status.** Every imported or changed program starts in *Needs review*, and only a curator's check of its rules makes it *Active*. This is how the design keeps the honesty principle: a program whose source changed is never matched on stale rules without being flagged.

A third state machine worth drawing during detailed design is the **account lifecycle**: Guest, then Registered (after consent), then Deleted (after UC10), with consents that can be withdrawn individually while the account stays active. A curator's sign-in adds one state: after the password, the session waits in "Second factor pending" until the code is verified.

B.5 Architectural design

The proposal is a three-tier web application, built in the Model-View-Controller style, with five subsystems, two separate data stores and a pipe-and-filter import pipeline.

Architectural design (also called system design) decides how the system is split into parts, how those parts talk to each other, and where they run. Good choices here make the rest of the work easier; poor ones are expensive to undo.

Key terms

Term	Meaning	Example here
Class	The smallest design unit: data plus behaviour	Program, EligibilityCriterion
Subsystem	A group of related classes with one clear responsibility	Matching (the rule engine and its classes)
Component	A replaceable, deployable piece of software that provides a subsystem's functionality	The matching library, the PDF generator
Service	A set of related operations a subsystem offers to others through its interface	evaluate(profile, programs) offered by Matching
Subsystem interface	The only door into a subsystem; others call it and never reach inside	Catalogue offers search(filters) and getProgram(id)
Node	A physical or virtual machine where components run	The user's browser, the application server, the database server

Proposed architecture

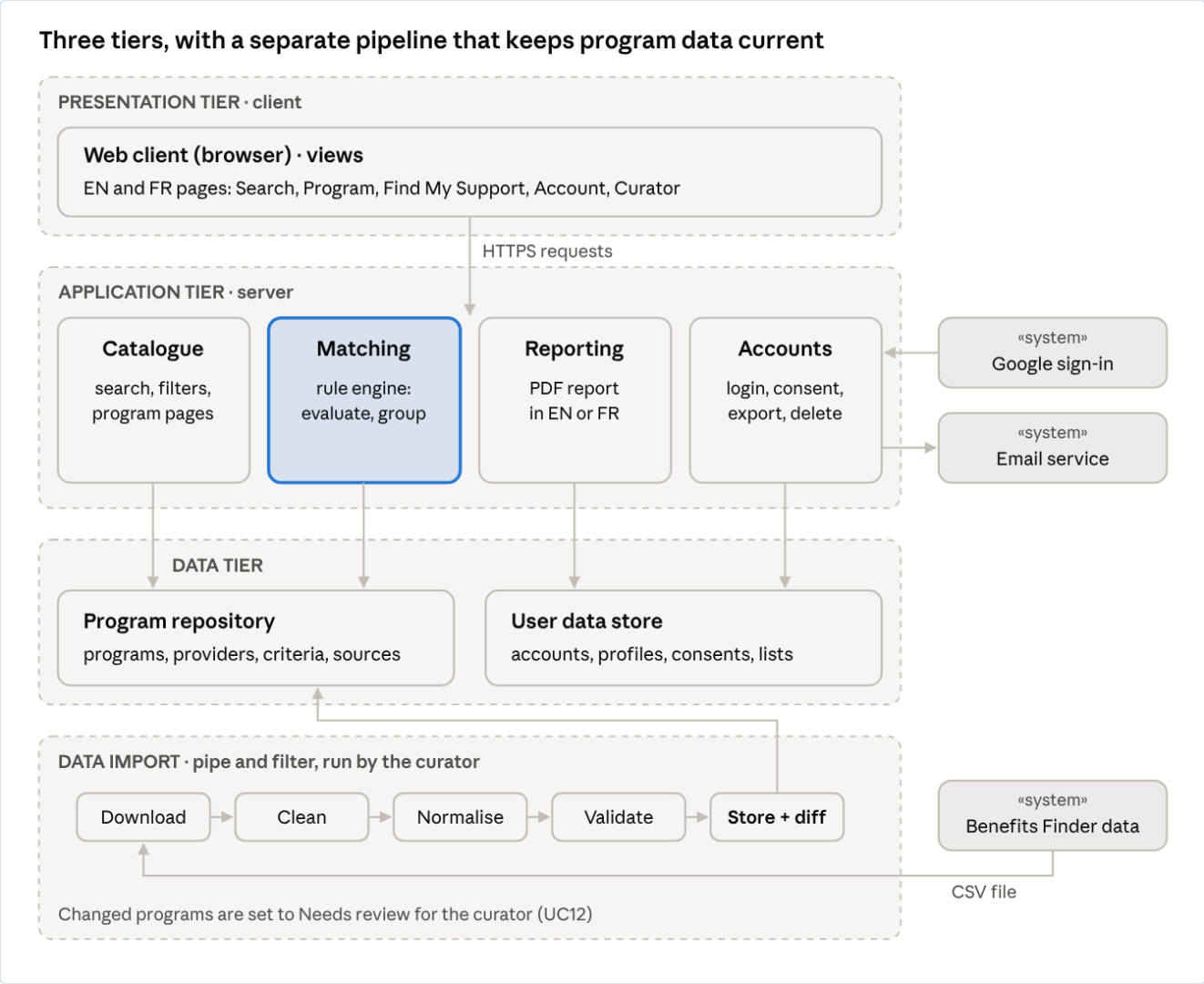


Figure 22. Proposed architecture · tiers, subsystems, repositories and external systems

Requests travel down from the browser to the application tier, which reads and writes the data tier. The import pipeline is separate: it is run by the curator, not by visitors, and it only writes to the program repository.

Subsystems and their interfaces

Subsystem	Responsibility	Interface (services offered)	Use cases
Catalogue	Programs, providers, filters and program pages	search(filters, language), getProgram(id)	UC1, UC2
Matching	Evaluate a profile against program rules and group the results	evaluate(profile): MatchResult[*], checklist()	UC3, UC13
Reporting	Produce the PDF report in the user's language	generateReport(user): PdfReport	UC8
Accounts and privacy	Sign-up, consent, login, roles and access control (with two-step verification for curators), profile, list, data export and deletion	register, login, verifySecondFactor, hasRole(user, role), saveProgram, exportData, deleteAll	UC4 to UC7, UC9, UC10, UC15
Data import and curation	Bring in the open dataset and maintain rules	runImport(file), listChanges(), updateRules(program)	UC11, UC12

Coupling and cohesion

- **High cohesion:** each subsystem does one job. Matching knows nothing about web pages, PDFs or accounts; it takes a profile and programs and returns results. That makes it the easiest part to unit test and to explain in a portfolio.
- **Low coupling:** subsystems talk only through their interfaces. Reporting asks Catalogue for program details rather than reading its tables directly, so the database can change without breaking reports.
- **Privacy by separation:** personal data lives in its own store, apart from public program data. Deleting a user touches only the user data store, and the program repository holds no personal information at all.

Architectural styles considered

Style	What it is	Fit for Maple Seed
N-tier (three-tier)	Presentation, application and data layers, each calling only the one below	**Chosen** as the overall structure
Model-View-Controller (MVC)	Views show data, models hold it, controllers handle user actions and connect them	**Chosen** inside the application: pages are views, entity classes are models, the control objects from B.3 are controllers
Client-server	Many clients send requests to one server	**Chosen**: browsers are clients of one web application
Repository	Subsystems share one central data store	**Chosen** for program data: Catalogue, Matching and Reporting all read the program repository
Pipe and filter	Data flows through a chain of independent steps, each transforming it	**Chosen** for data import: download, clean, normalise, validate, store; each step testable on its own
Peer-to-peer	Nodes act as both client and server with no central server	Not suitable: we need one trusted, curated source of program data

Deployment (nodes)

Node	What runs there
User's device	A web browser showing the views; no installation
Application server	The web application: controllers and the five subsystems
Database server	The program repository and the user data store, ideally in two separate databases or schemas
External services	Google sign-in, an email service, the open data portal

Hosting in Canada is preferred, so personal data does not leave the country (this also simplifies the privacy notice, which must name any transfer outside Québec or Canada).

Technology stack (decided 30 October 2026)

Layer	Choice	Why
Frontend	React (JavaScript), running in the browser	Browsers run only HTML, CSS and JavaScript; the prototype's HTML and CSS can be reused as components
Backend	Python with FastAPI	The language being learned; quick to start; produces a JSON API that fits the three-tier design
Database	PostgreSQL	The relational schema maps directly from the class diagram
Tests	pytest, run by GitHub Actions on every commit	Less code per test than unittest

How the parts connect: the React frontend sends the user's answers to the API (for example, POST /match), the backend runs the domain classes and returns the results as JSON, and the frontend draws the screen. The look comes from CSS, not from the language.

Alternative considered: Django rendering the pages on the server. It means one language and fewer parts, but a less dynamic interface and less separation between tiers. Not chosen, because the project aims to show a layered architecture with an API.

B.6 From design to a working MVP

Build from the inside out: validate the model with real programs, implement the domain with unit tests, then add the database, the API and the screens. Deliver in slices, starting with the public product without accounts.

Implementation phases

#	Phase	Deliverable	Done when
1	Preparation	MVP scope, stack recorded, repository with automated tests	A commit shows a passing test on GitHub
2	Validate the model with real programs	10 to 15 real programs written as rules; model version 2	Every criterion fits the model or is recorded as out of scope
2b	Validate the problem (in parallel)	Notes from three to five conversations with business owners	The pain hypotheses in A.8 are confirmed or revised
3	Object design	Detailed class diagram and interfaces	Each class can be written without inventing anything
4	Domain and matching engine	Plain Python classes with passing unit tests	Tests cover Match, Maybe and Not a match
5	Persistence and import	PostgreSQL schema, migrations, first import	Phase 4 tests pass with data from the database
6	API	Search, program and matching endpoints	Find My Support can be completed through HTTP calls alone
7	Public screens	Find My Support, search and program page in English and French	Someone outside the project uses it without help: **MVP 1**

#	Phase	Deliverable	Done when
8	Accounts, saved list and report	Sign-in, consent, profile, list with checklist, PDF report	Data export and deletion work end to end
9	Curator console	Roles, invitations, two-step verification, rule editor	Only curators reach the console
10	Quality and release	Accessibility review, deployment, case study	The demo is public and reviewed

MVP 1 scope (MoSCoW)

MoSCoW sorts requirements into Must (the release fails without it), Should (important, but the release still works), Could (nice to have) and Won't (left out for now, on purpose). Applied to the use cases in B.2, it gives the scope of MVP 1 and the order of the later phases:

Priority	Use cases	Why
Must (MVP 1)	UC1 Search programs, UC2 View program details, UC3 Find my support	The core of the product, answering pain points 1, 2, 3 and 5. Works without an account.
Must, minimal version (MVP 1)	UC11 Import program data	A command run by the curator to load programs.json into the database, with no screen.
Should (phase 8)	UC4 Create account to UC10 Delete my data	They come together: once accounts exist, downloading and deleting personal data become legal obligations (PIPEDA, Law 25).
Could (phase 9)	UC12 Maintain eligibility rules, UC15 Log in as curator	While the author is the only curator, rules are edited in the data file; the console comes later.
Could (later)	UC13 Get recommendations	Depends on a saved business profile.
Won't (for now)	UC14 Subscribe to program news	Needs an email service and CASL consent; left out of the current plan.

The Release column in the use case list in B.2 follows this table.

What each phase involves

1. **Preparation.** Classify the 13 use cases with MoSCoW (Must, Should, Could, Won't); only the Must cases enter MVP 1, which are UC1, UC2, UC3 and a minimal UC11 (see MVP 1 scope below). Record the stack choice (B.5). Create the repository with its folder structure, a first test and GitHub Actions running the tests on every commit.
2. **Validate the model with real programs.** Choose 10 to 15 real federal and Ontario programs. For each, record from the official website the provider, amount, type of support and every eligibility criterion in a spreadsheet. Fit each criterion to a criterion subclass and record what does not fit; that list is the most valuable result of the phase. Update the class diagram (SavedItem, new criterion types, joint providers) and write the programs to a programs.json file. In parallel, talk with three to five business owners, showing the prototype, to test the pain hypotheses before code is written.
3. **Object design.** Exact types, visibility, method signatures and navigability for every class. Interfaces such as ProgramRepository (findAll, findById), implemented first over programs.json and later over the database. Contracts for key methods (for example, evaluate never returns Match when a criterion gave Does not fit). Design patterns where they help: Strategy for criterion types, Facade for subsystem interfaces, Adapter for the Benefits Finder file format, Template Method for the import steps.
4. **Domain and matching engine.** Implement the entity classes, the six criterion subclasses, Program.evaluate() and the MatchResult grouping as plain Python classes, with no database, web or screen code, loading programs from programs.json. Write pytest unit tests: at least one Fits, To confirm and Does not fit per criterion type, and full cases per program from phase 2.
5. **Persistence and import.** Derive the schema from the model (see "From relationships to code and database" in B.3), choose how to store criterion inheritance, write migrations, implement ProgramRepository over PostgreSQL without changing the engine, and import programs.json.
6. **API.** FastAPI endpoints GET /programs (with filters), GET /programs/{id} and POST /match; input validation; English and French text; API tests.
7. **Public screens.** React screens from the prototype: home, Find My Support with grouped results and reasons, Search by Filters and program details, in English and French from the start, with basic accessibility (keyboard, contrast, labels).
8. **Accounts, saved list and report.** Email and Google sign-in, consent, saved profile, SavedList with SavedItem and checklist, PDF report, data export and deletion, with the PIPEDA and Law 25 requirements.
9. **Curator console.** Roles, invitation-only accounts, two-step verification, rule editor and the Needs review → Active → Closed workflow.
10. **Quality and release.** WCAG 2.1 AA review, deployment, a second round of conversations with owners and a portfolio case study.

Why the domain comes before the database

The **domain** is the subject the software deals with: programs, providers, criteria, businesses and eligibility. It exists independently of screens and storage; a funding adviser with no computing background would understand every term in it. Domain classes are the class diagram written as code. Tables, endpoints and buttons are technical concerns, not domain.

The code and the database schema are two translations of the same model (Bruegge and Dutoit, chapter 10, *Mapping Models to Code*). The model can still be wrong, and errors appear when the rules are written and tested against real programs, as the missing SavedItem did. Changing a class is cheap; changing tables that already hold data is not. So while the model settles, programs live in a plain file that is easy to edit, and the database is built once the classes are stable. Unit tests on these plain classes run in milliseconds, document the rules, and act as a safety net when a rule changes later.

This order suits Maple Seed because its difficulty is in the rules. A system that is mostly data entry often starts from the database instead.

Three principles guide the plan:

- **Inside out.** Rules first, then what stores them (database), exposes them (API) and shows them (screens); each layer starts once the one inside it is tested.
- **Biggest risk first.** The main risk is not technical: real criteria may not fit the model. Phase 2 finds out on paper, before any code.
- **Deliver in slices.** MVP 1 is a complete product without accounts.

Testing and traceability

Each requirement should be traceable through the models to code and tests. A test is written from each use case's entry condition, flow and exit condition.

Requirement	Use case	Subsystem and classes	Example test
Show why a program matches	UC3	Matching: Program, EligibilityCriterion, MatchResult	A profile in Ontario against an "Ontario only" criterion returns Fits with the reason text
Group results correctly	UC3	Matching: MatchResult	One Does not fit gives No; only To confirm gives Maybe; all Fits gives Match
Delete all personal data	UC10	Accounts: User, BusinessProfile, Consent, SavedList	After deleteAll(), no record of that user remains in the user data store
Flag changed programs	UC11	Data import: ImportController	A changed program in the new file ends in Needs review
Bilingual report	UC8	Reporting: PdfReport	A report requested in French has every heading and program name in French
Only curators reach the curator console	UC15	Accounts: User.role, hasRole()	An Owner account that opens the program catalogue is refused; a Curator account without the second factor is refused

Test levels: **unit tests** for each criterion type and the grouping rule; **integration tests** for each subsystem interface; **acceptance tests** that walk through each use case as a user would.

Deployment and maintenance

- **Configuration management:** version control from the first line of code, with small, described commits.
- **Content maintenance:** a regular import of the Benefits Finder dataset (for example, monthly), with every changed program reviewed before it is matched again.
- **Monitoring the promise:** the "last checked" date on each program shows visitors how current the information is.

Appendix: glossary and sources

Glossary

Term	Meaning
SME	Small and medium-sized enterprise. In the statistics used here, small = 1 to 99 employees, medium = 100 to 499
ISED	Innovation, Science and Economic Development Canada, the federal department that publishes the Business Benefits Finder and the small business statistics
MVP	Minimum viable product: the smallest version that is useful and can be tested with users

Term	Meaning
UML	Unified Modeling Language, the standard notation for software models (use case, class, sequence and state diagrams)
PIPEDA	Personal Information Protection and Electronic Documents Act, the federal privacy law for the private sector
Law 25	Québec's modernised Act respecting the protection of personal information in the private sector
CASL	Canada's Anti-Spam Legislation, which requires consent before sending commercial email
WCAG	Web Content Accessibility Guidelines; level AA is the common target
CIPO	Canadian Intellectual Property Office, where trademarks are searched and registered
CIRA	Canadian Internet Registration Authority, which manages .ca domains
IRAP, SR&ED	Two federal programs used as examples: the Industrial Research Assistance Program and the Scientific Research and Experimental Development tax incentive

Sources

The statistics in Part A were read from these federal publications in October 2026 and should be rechecked before reuse:

- Innovation, Science and Economic Development Canada, *Key Small Business Statistics 2025*. Business counts to December 2024, employment 2024, GDP shares 2022.
- Innovation, Science and Economic Development Canada, *Credit Conditions Survey 2024*. 1,825 small businesses surveyed January to April 2025, covering financing sought in 2024.
- Innovation, Science and Economic Development Canada, *Borrower Awareness and Satisfaction Study 2023–2024*, Canada Small Business Financing Program. Fieldwork November 2023 to January 2024.
- *Business Benefits Finder* dataset, Open Government portal, published under the Open Government Licence – Canada.

Method reference: Bernd Bruegge and Allen H. Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns, and Java*, 3rd edition, Pearson, 2009.

Companion artifacts: the Maple Seed Funding Guide design canvas (screens, style guide, brand record) and the project presentation (20 slides).